
CIS437 - Cloud Computing

– Choosing Cloud Services –
IaaS, PaaS, SaaS, and Serverless

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Last Class

- What makes an application cloud-ready?
- APIs and cloud automation
- Regions, zones, and fault tolerance
- Monitoring and migration
- Scalability vs. elasticity
- Managed Instance Groups and autoscaling

Agenda

- Cloud service models
- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Serverless / Functions as a Service
- Software as a Service (SaaS)
- Compare the models
- Choosing the right service

Major Cloud Service Models

- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Software as a Service (SaaS)

The key difference is how responsibility is divided between you and the cloud provider.

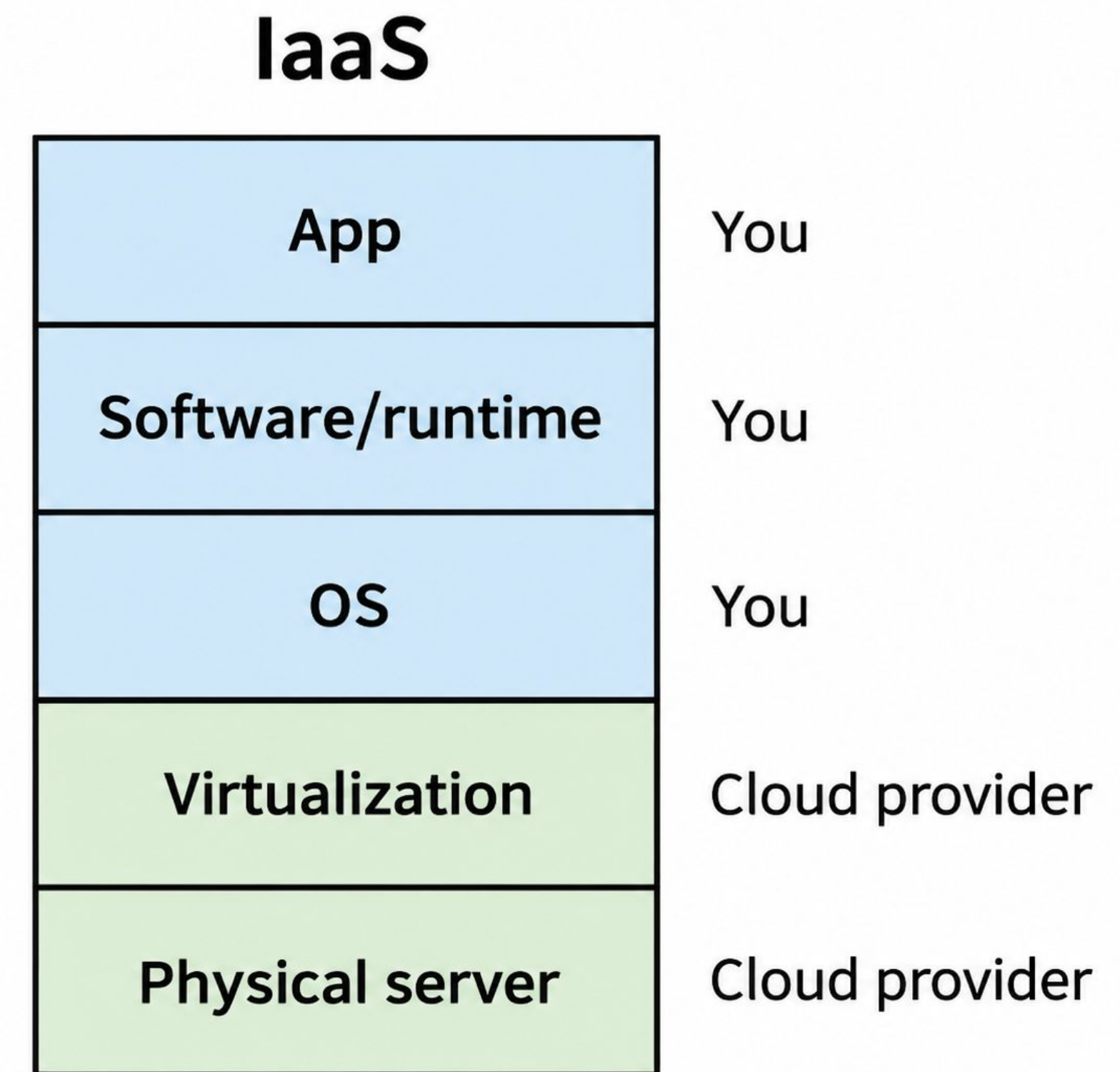
Who Manages What?

Layer	IaaS	PaaS	SaaS
Application	You	You	Provider
Runtime / middleware	You	Provider	Provider
Operating system	You	Provider	Provider
Virtualization	Provider	Provider	Provider
Hardware / networking	Provider	Provider	Provider

More control ← IaaS — PaaS — SaaS → Less infrastructure to manage

Infrastructure as a Service (IaaS)

- Cloud provider supplies virtualized computing infrastructure
- Typically includes:
 - Virtual machines
 - Storage
 - Networking
- You still manage:
 - Operating system
 - Runtime / software
 - Application
- No need to own and maintain the physical server



Example: Google Compute Engine

IaaS: What Are We Responsible For?

VM configuration

- Machine type / resources
- Operating system
- Image or template
- Storage



So when might we actually want this level of control?

Security and networking

- Who can access the VM?
- Firewall rules
- Network configuration

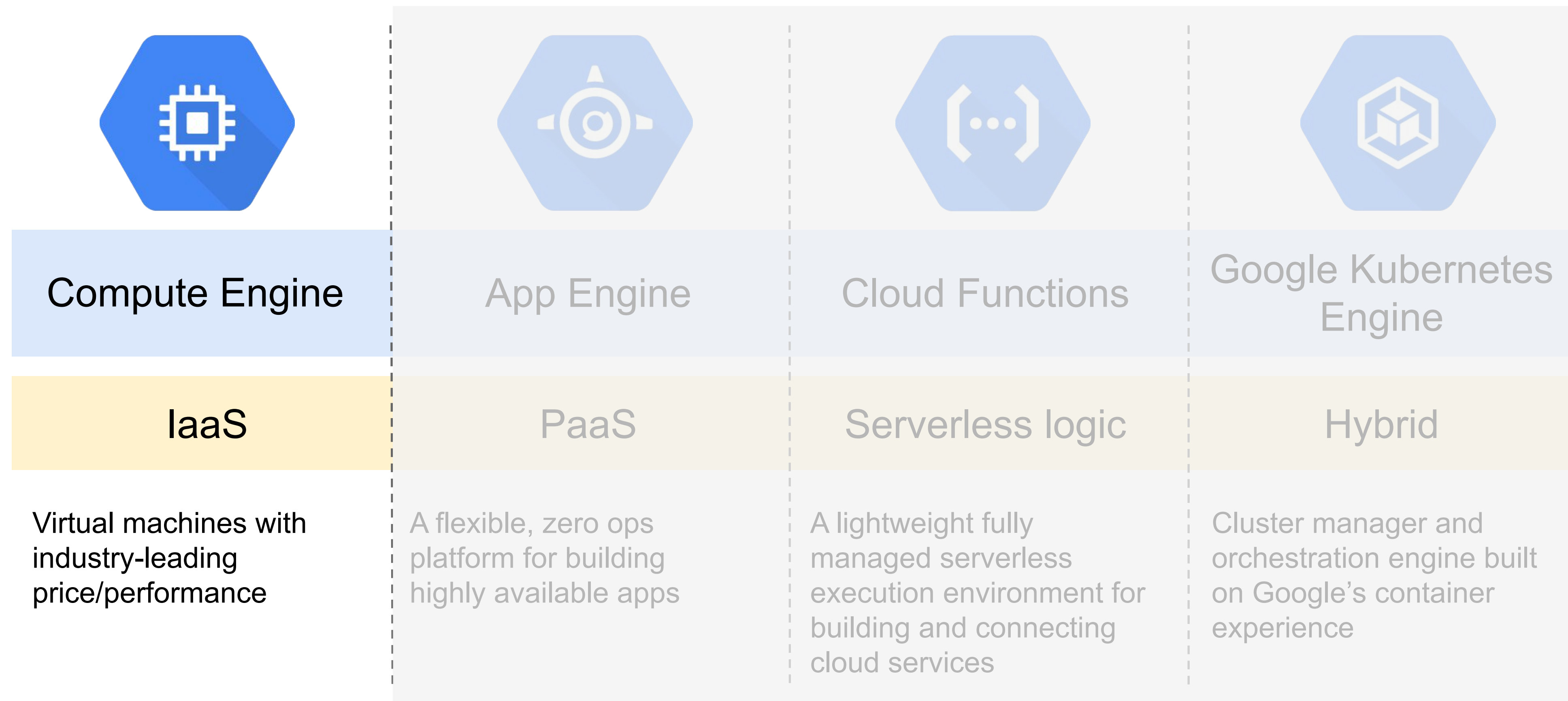
Operations

- Scaling
- Location / region / zone
- Monitoring
- Cost



More control also means more management responsibility.

Where Compute Engine fits within Google Cloud



Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program [Website](#).

Compute Engine is an infrastructure-centric solution

- IaaS virtual machines
- Linux and Windows
- Predefined or custom machine types
- Durable storage options
- Flexible networking and security
- Scale from small workloads to large deployments
- Pay for cloud resources as you use them



Choose a VM for the workload

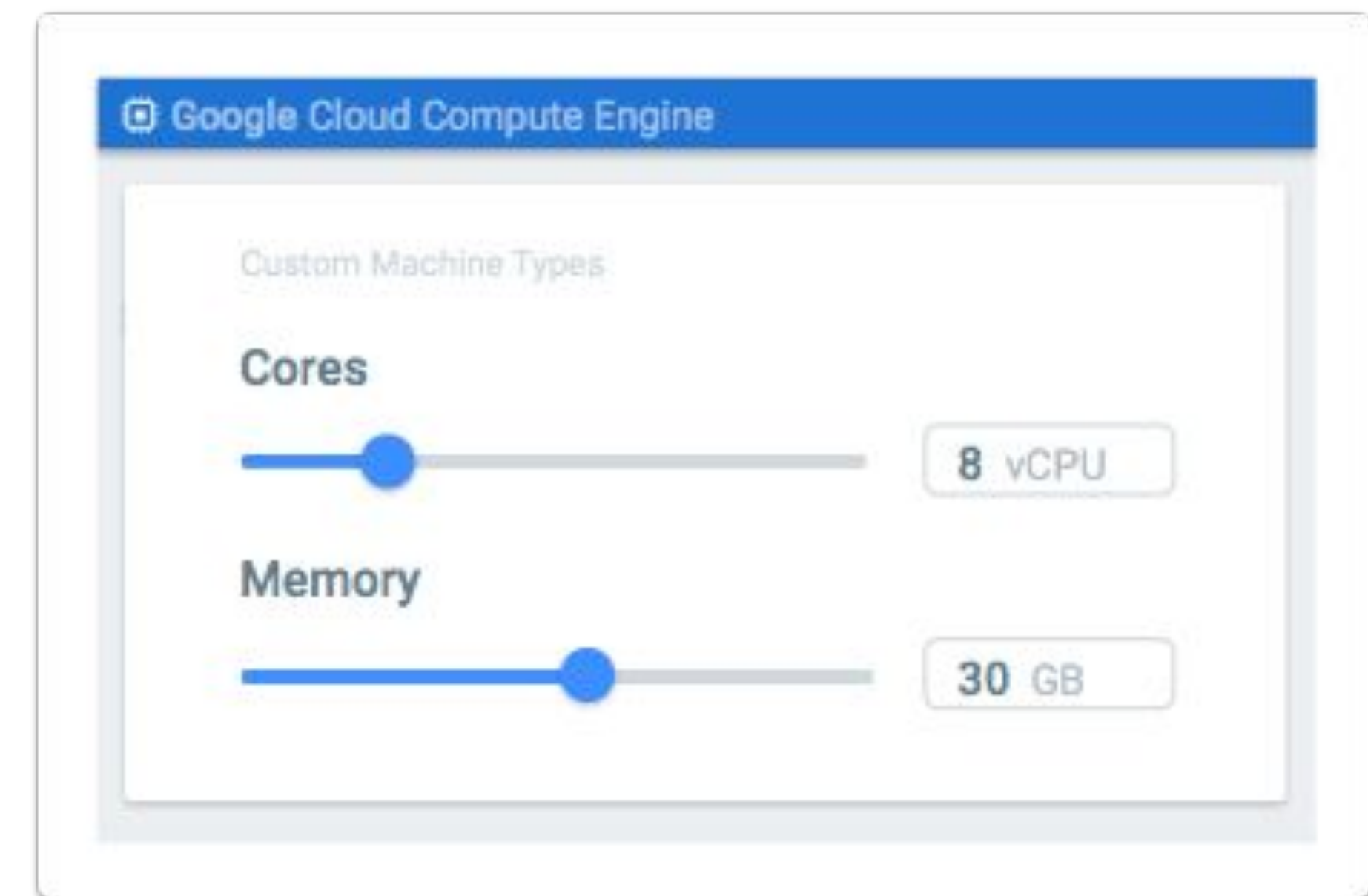
General-purpose — balanced CPU and memory

Compute-optimized — CPU-intensive workloads

Memory-optimized — memory-intensive workloads

Accelerator-optimized — GPUs / AI / HPC

Custom machine types — choose CPU and memory when supported



Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program [Website](#).

Storage for Compute Engine

Storage type

Main idea

Hyperdisk / Persistent Disk

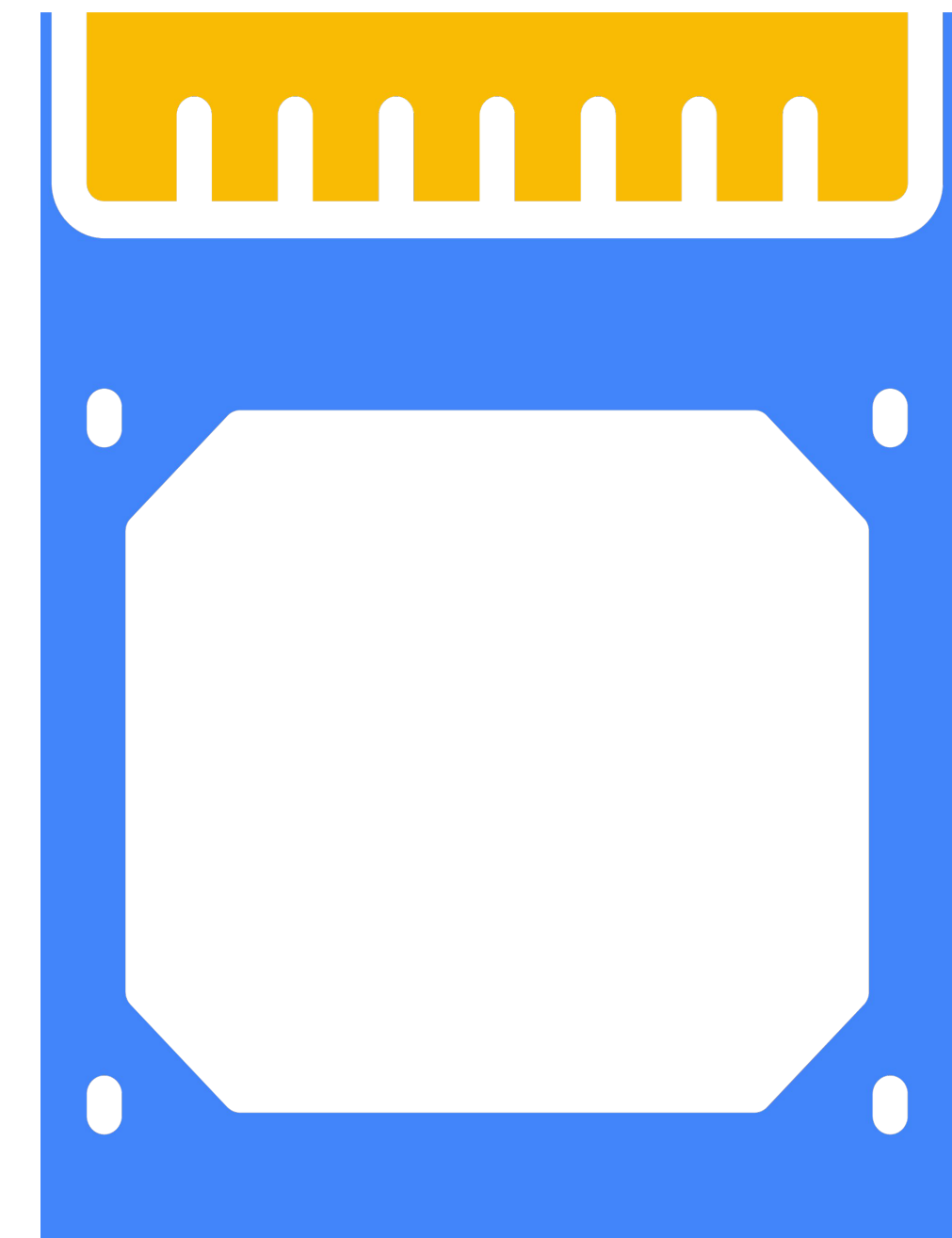
Durable, network-attached block storage

Local SSD

Very fast, low-latency storage physically attached to the host

IOPS: Input/output operations per second — number of read/write operations per second

- Persistent Disk data can remain after a VM is stopped or deleted, depending on lifecycle settings, and disks can be detached/re-attached or snapshotted.
- Persistent Disk performance depends on **disk size** and the **VM machine type / vCPU limits**.
- Local SSD has much higher performance and lower latency, but it is **temporary, non-durable storage**.
- Hyperdisk is Google's newer durable block-storage family, with performance that can be provisioned more independently from capacity. Google currently recommends Hyperdisk when the machine series supports it.

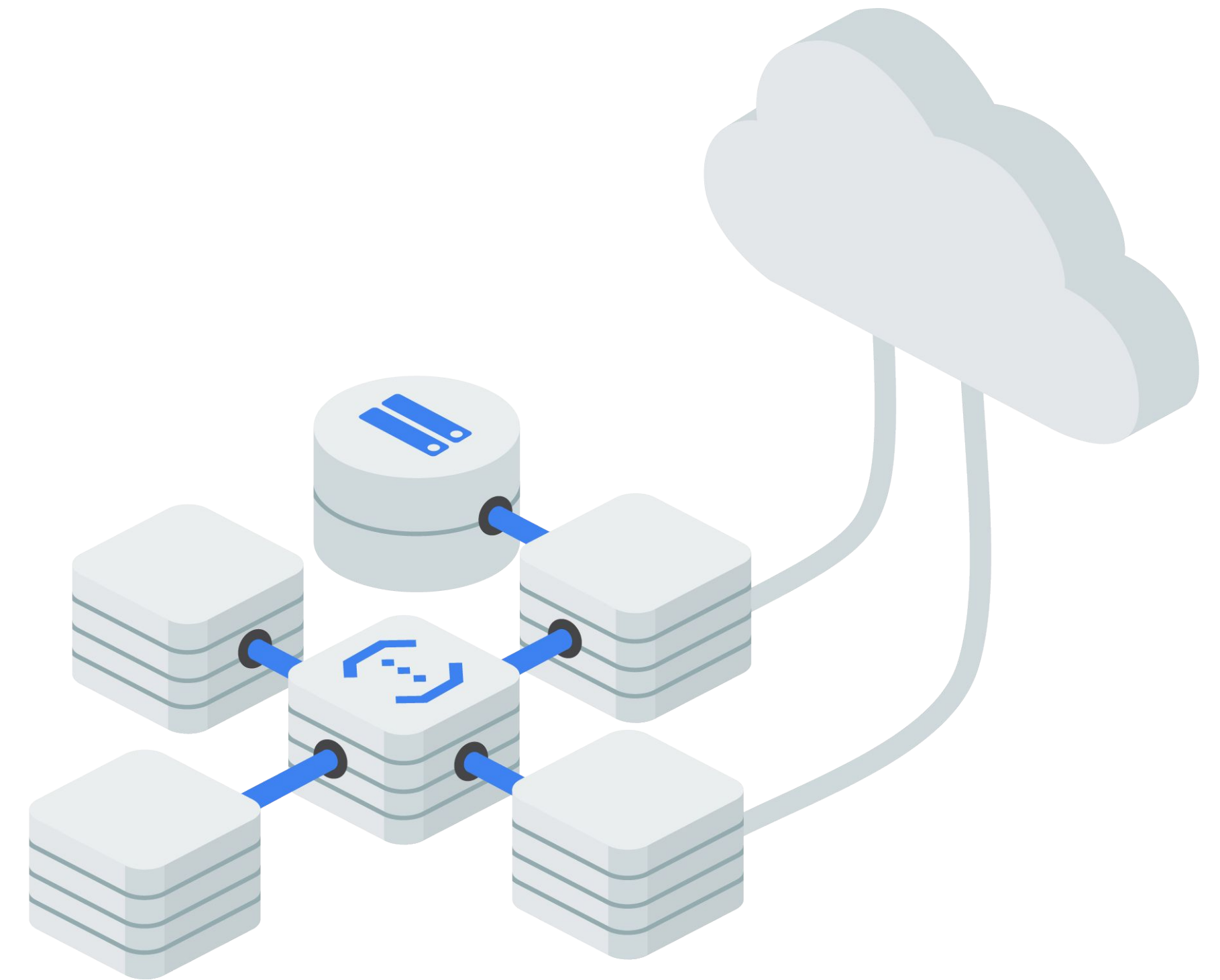


[Cloud console](#)

Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program [Website](#).

Networking for Compute Engine

- VM network interfaces connect instances to a **Virtual Private Cloud (VPC) network**
- **Subnets** organize IP address ranges by region
- **Firewall rules** control allowed traffic
- **Routes** determine where traffic goes
- **Load balancing** distributes traffic across backends



IaaS Strengths / Responsibilities

Strengths	Responsibilities
High control	Configure the VM
Flexible OS/software	Patch and maintain OS
Many machine/storage choices	Configure security/firewalls
Supports custom/legacy workloads	Monitor and scale
Infrastructure on demand	Manage cost and resources

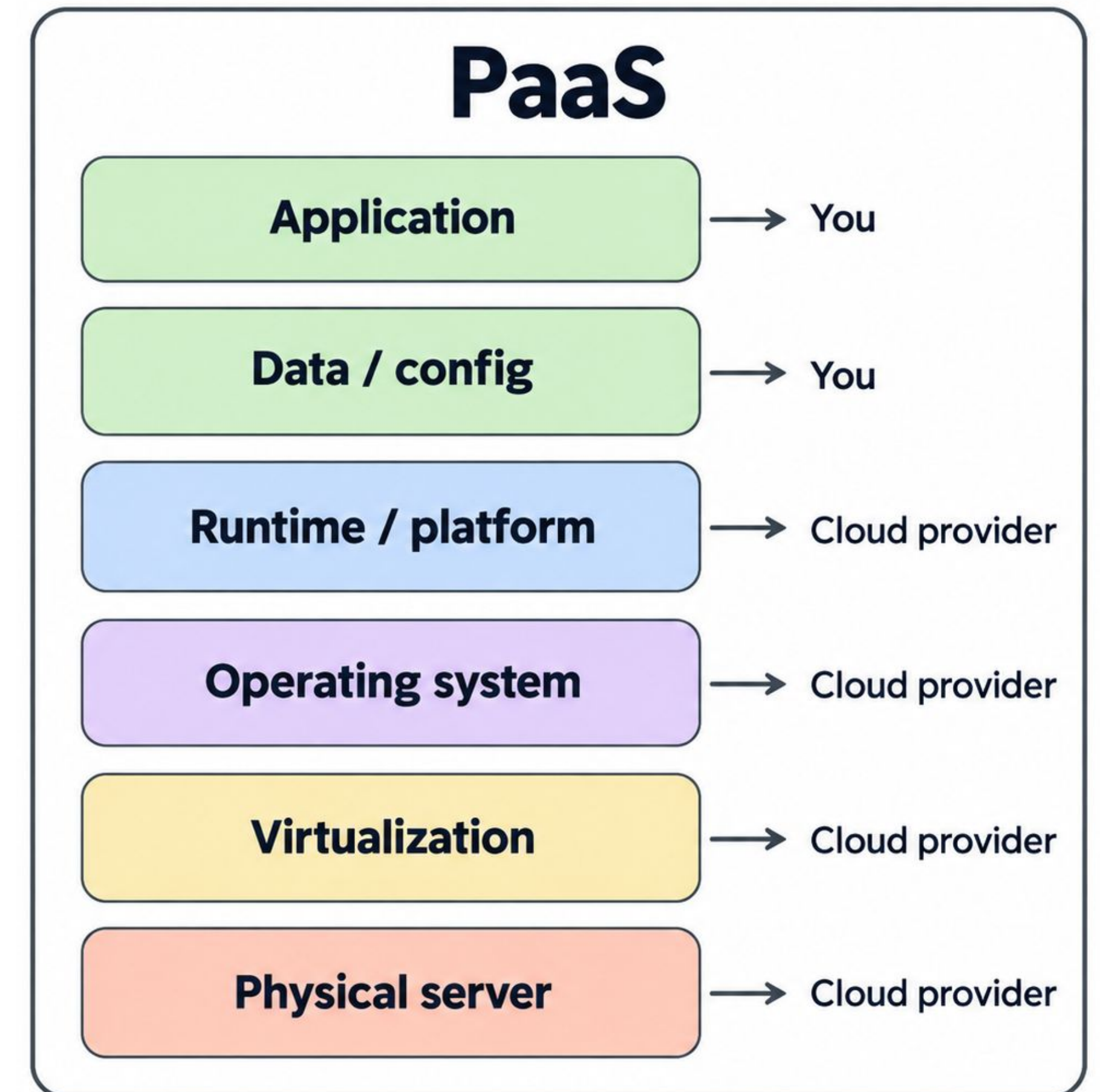
IaaS = more control + more responsibility

Platform as a Service (PaaS)

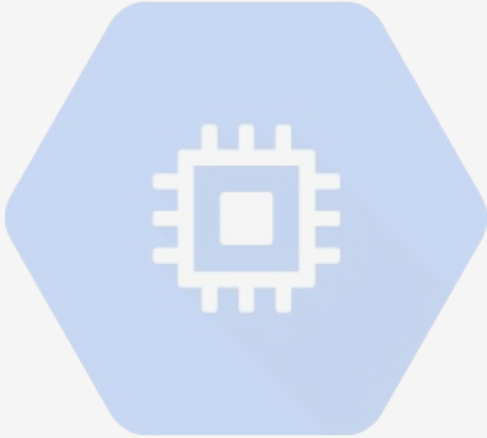
- Cloud provider manages the **infrastructure, operating system, and runtime/platform**
- You focus mainly on:
 - application code
 - application configuration
 - data
- Deploy applications without directly managing VMs
- Built-in platform features may include scaling, deployment, monitoring, and logging

Example: Google App Engine

PaaS = focus on the application, not the server



App Engine: A Managed Application Platform

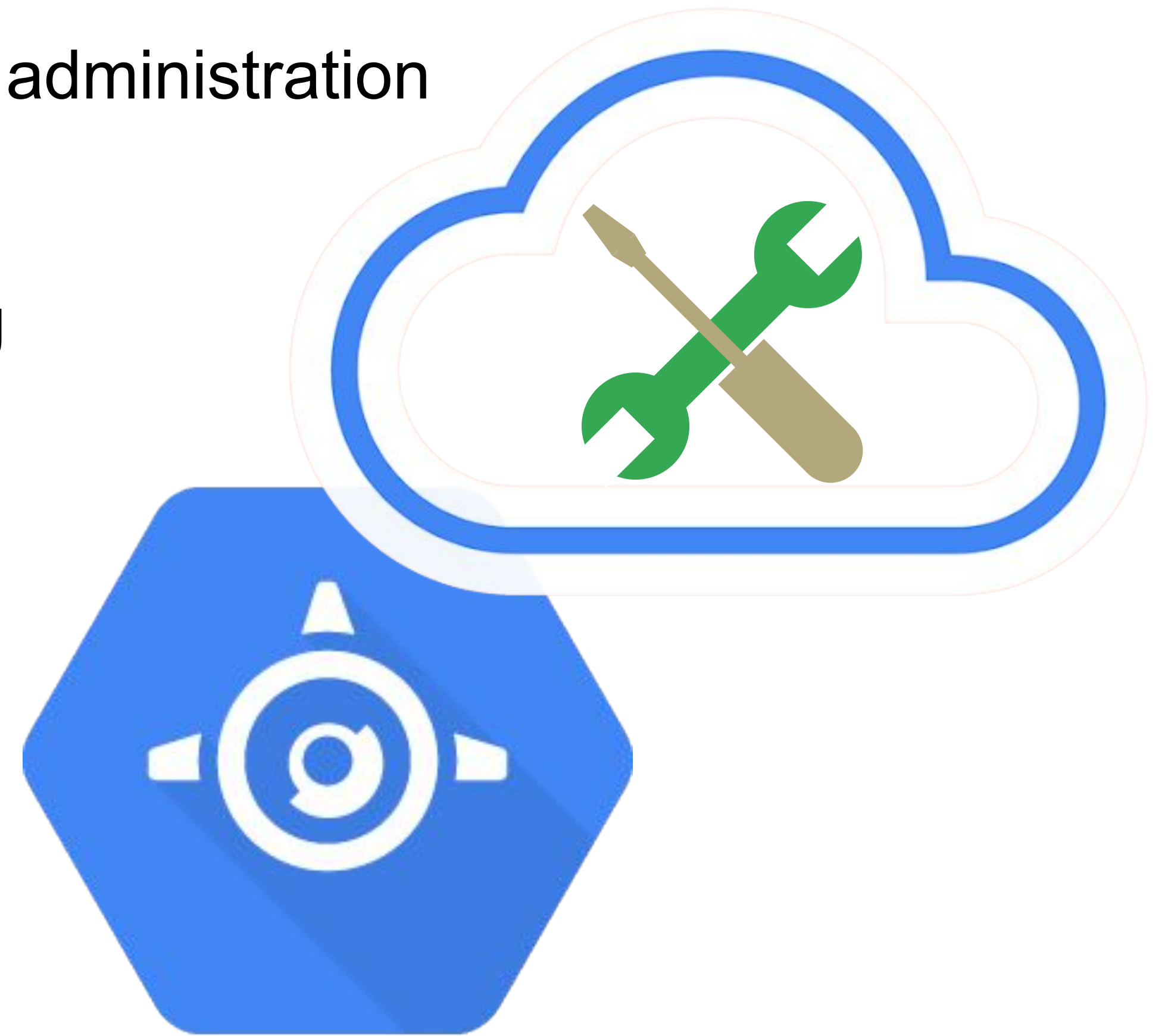
			
Compute Engine	App Engine	Cloud Functions	Google Kubernetes Engine
IaaS	PaaS	Serverless logic	Hybrid
Virtual machines with industry-leading price/performance	A flexible, zero ops platform for building highly available apps	A lightweight fully managed serverless execution environment for building and connecting cloud services	Cluster manager and orchestration engine built on Google's container experience

Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program [Website](#).

App Engine is a platform-centric solution

- Type of PaaS
- Google manages the underlying infrastructure and OS
- Focus on application development instead of server administration
- Supports multiple language runtimes
- Built-in deployment, monitoring, logging, and scaling
- Automatic scaling based on traffic

**No direct server management;
Deployment is handled through the platform**



App Engine: Standard vs. Flexible

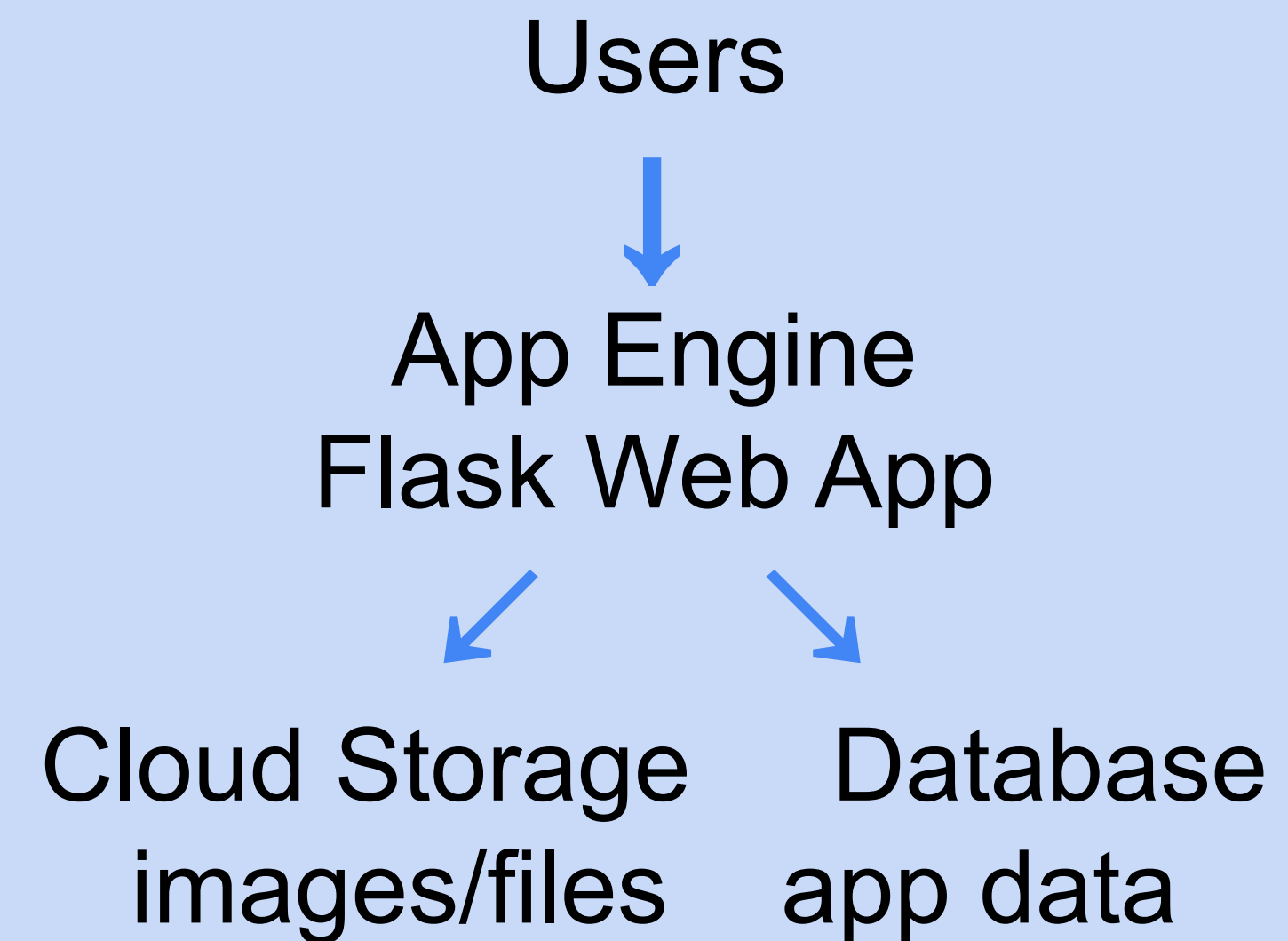
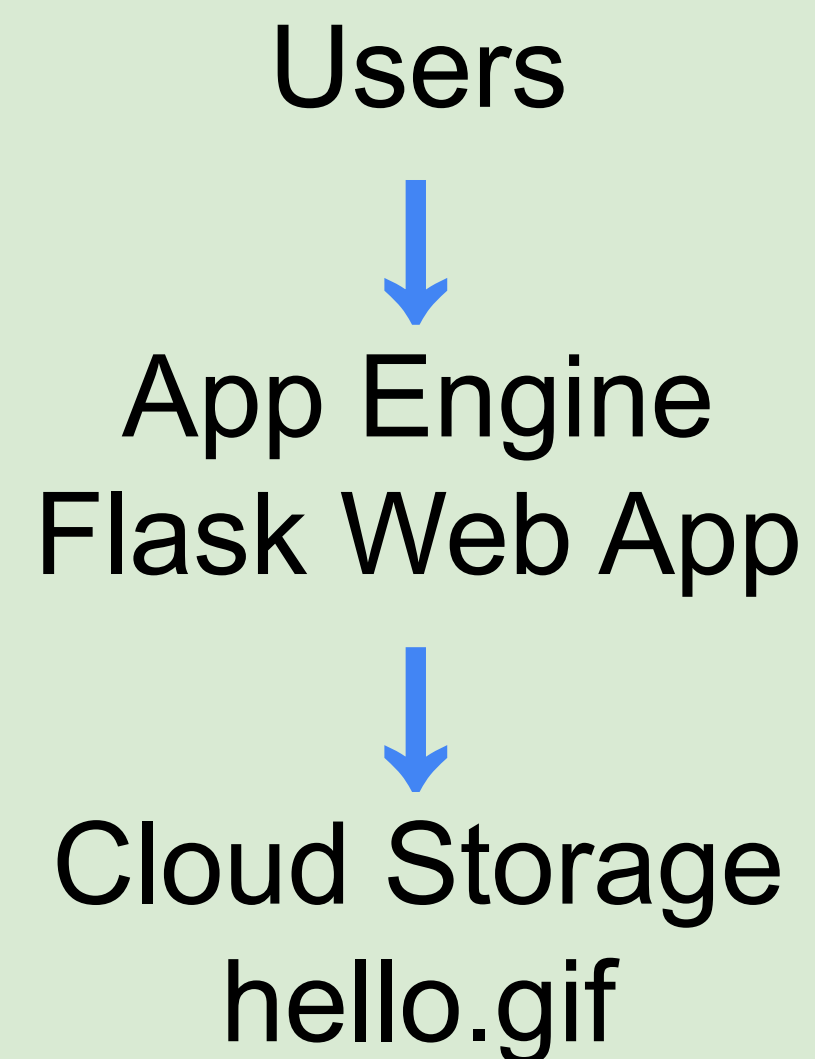
Standard environment

- Fully managed, sandboxed runtime
- Fast startup and rapid scaling
- Can scale to zero
- Supported language runtimes
- No SSH access
- Limited runtime customization
- Local writable /tmp only

Flexible environment

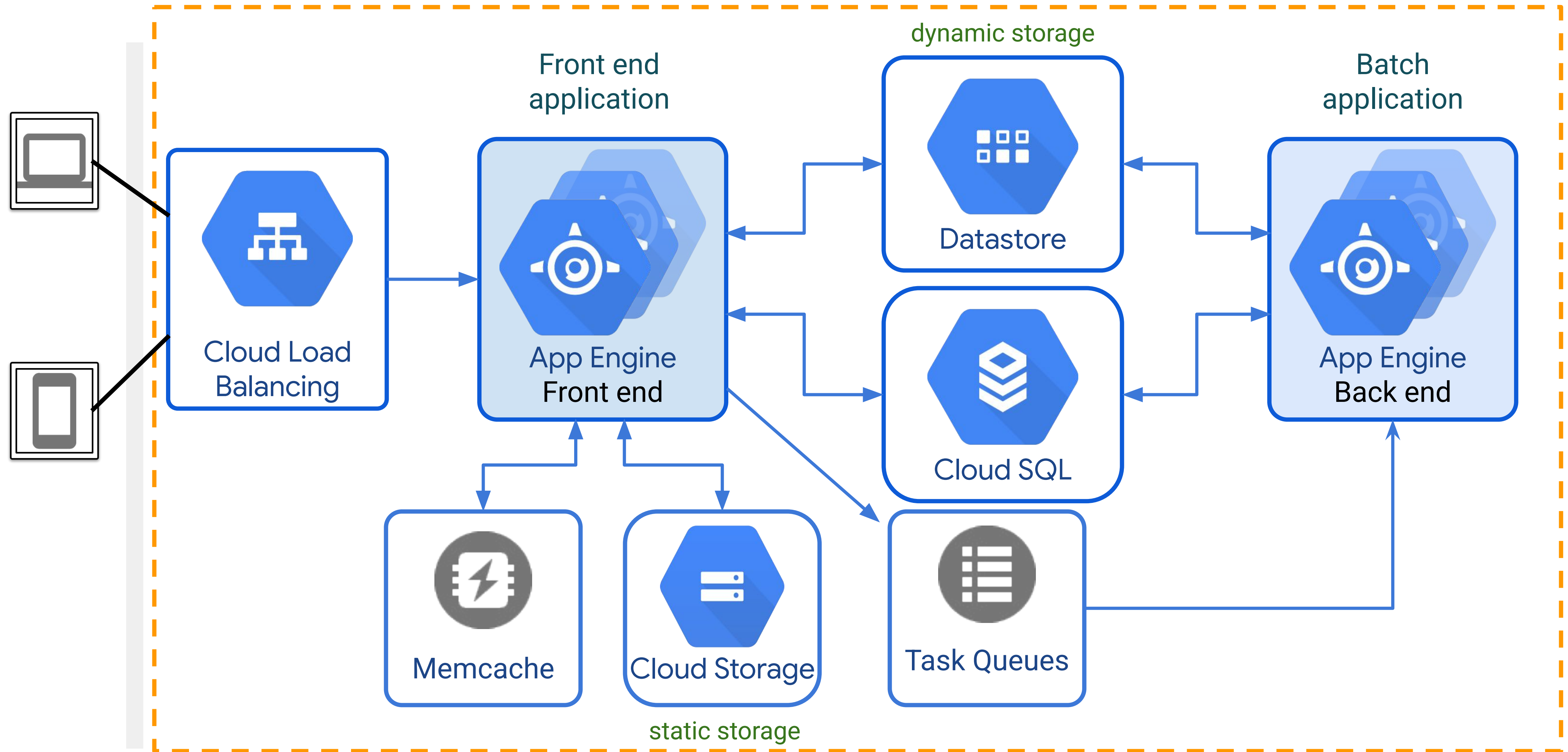
- Docker containers on Compute Engine VMs
- More customization, slower startup
- Cannot scale to zero; at least 1 instance
- Supported or custom runtimes
- SSH/root access can be enabled
- Customize runtime with Docker
- Writable local disk, but **ephemeral**

From Our Flask Demo to Architecture



A cloud application is often a collection of managed services working together.

A Modern App Engine Architecture



Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program [Website](#).

Legacy App Engine services and modern alternatives

App Engine addresses key application needs



Integration with
storage and
database services



Automatic scaling



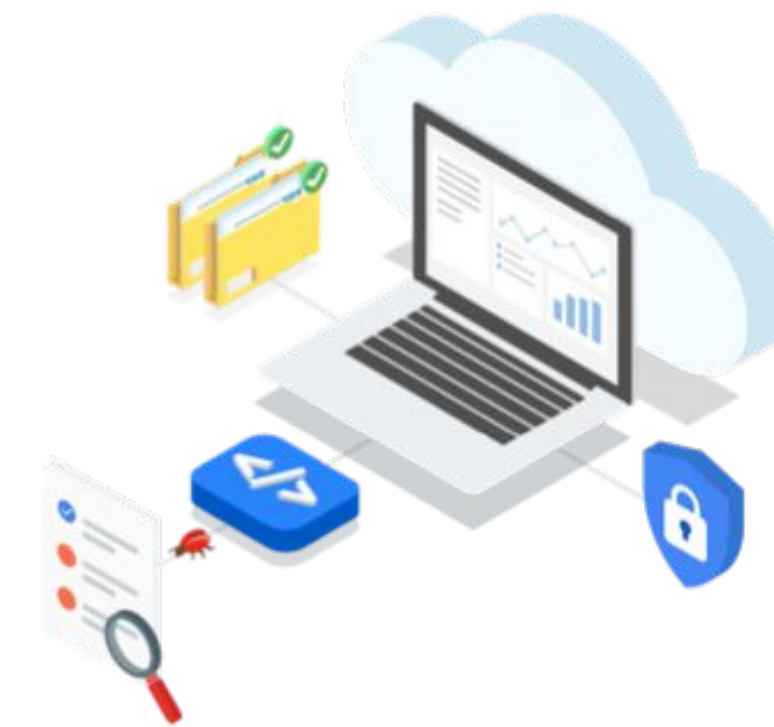
Traffic distribution
/ routing



App versioning



Monitoring and
logging



Security

Functions as a Service (FaaS)

Deploy a **function**, not an entire server/application

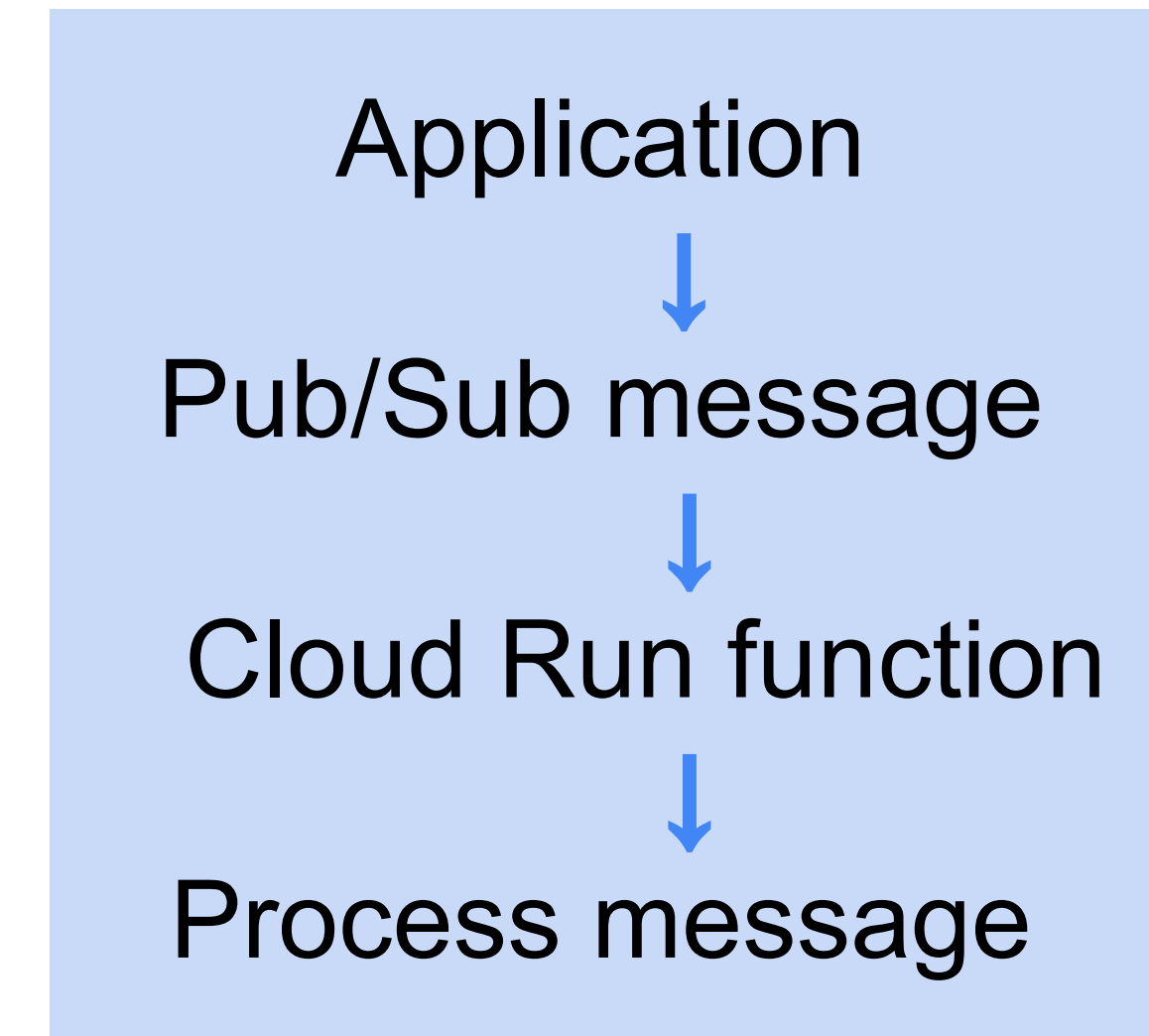
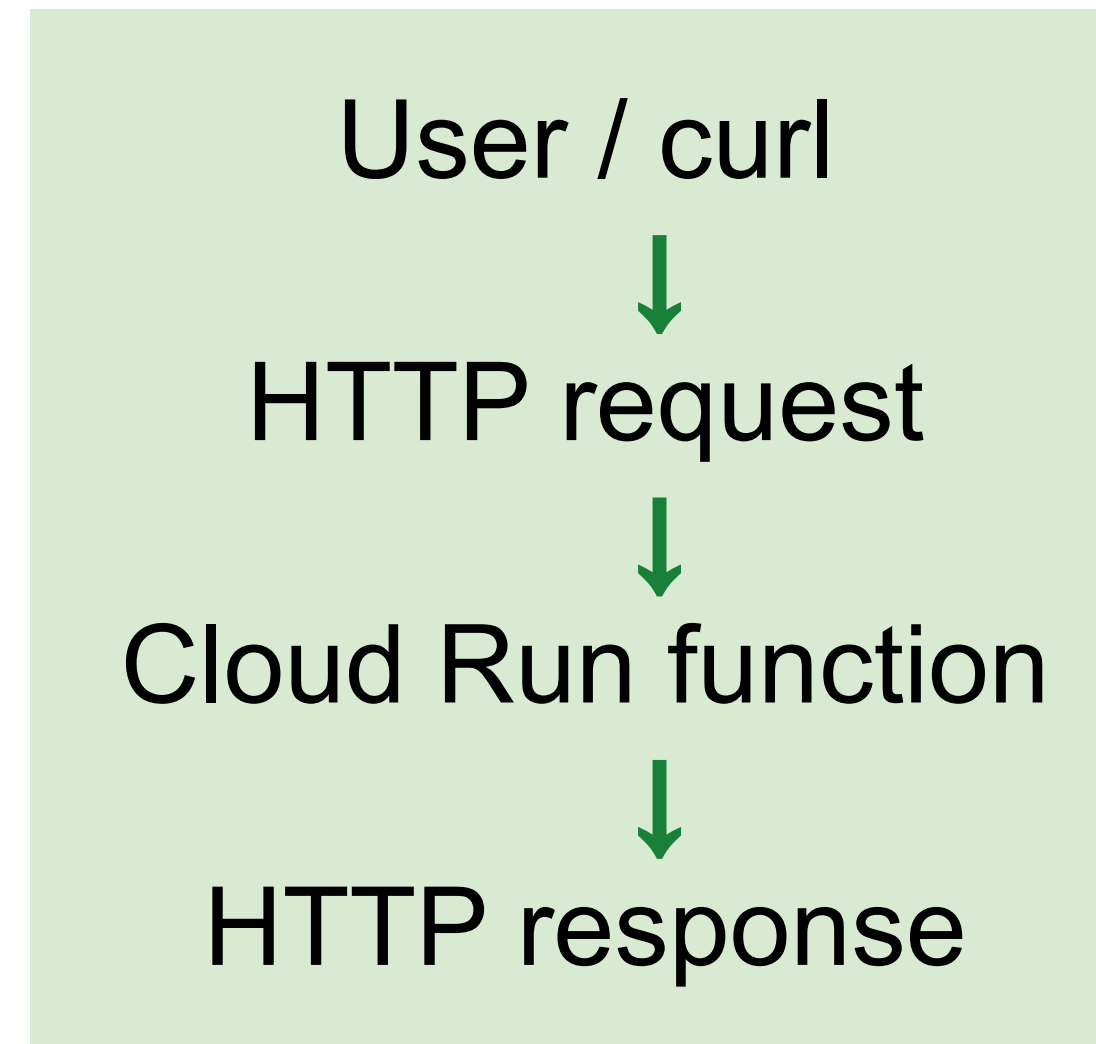
Function runs when it is **triggered**

Common triggers:

- HTTP request
- Pub/Sub message
- Cloud Storage event
- scheduled/event-driven action

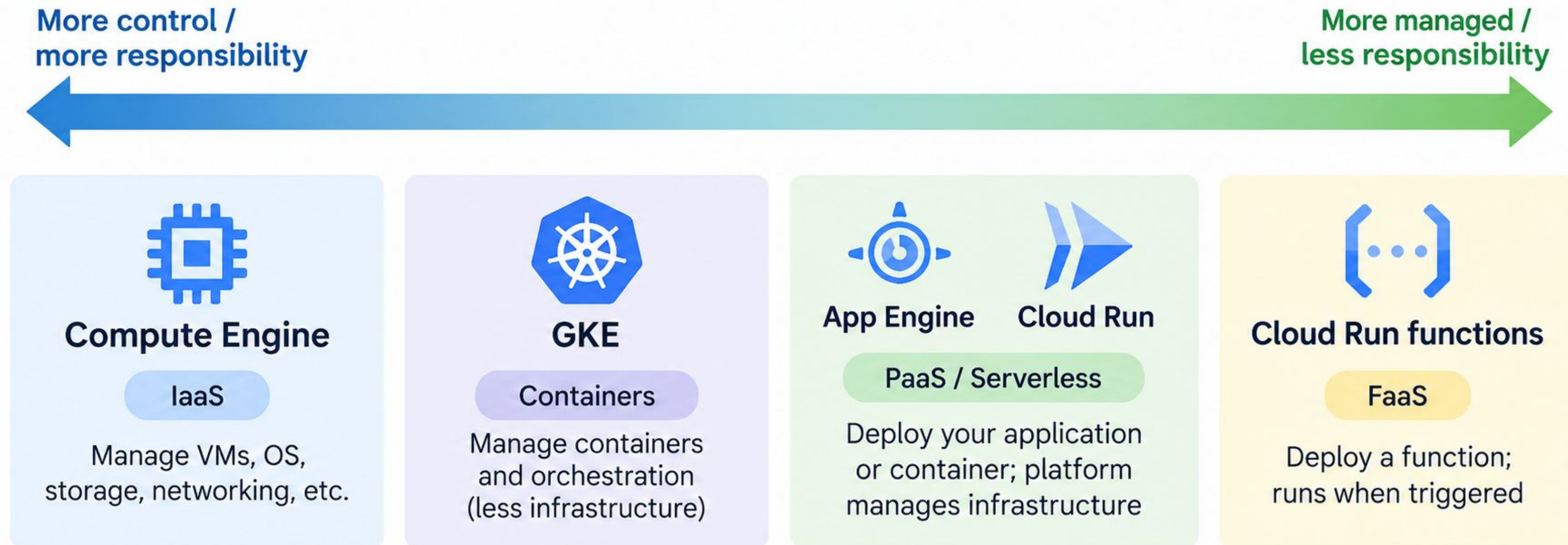
Cloud provider manages servers and scaling

Examples: Google Cloud Run functions, AWS Lambda, Azure Functions



FaaS is a serverless pattern: write the function, define the trigger, let the platform run it.

A Spectrum of Control and Management



These categories overlap; cloud services exist on a spectrum rather than fitting perfectly into boxes.

Software as a Service (SaaS)

Provider delivers a complete application

Provider manages:

- infrastructure
- operating system
- runtime
- application software
- updates and maintenance

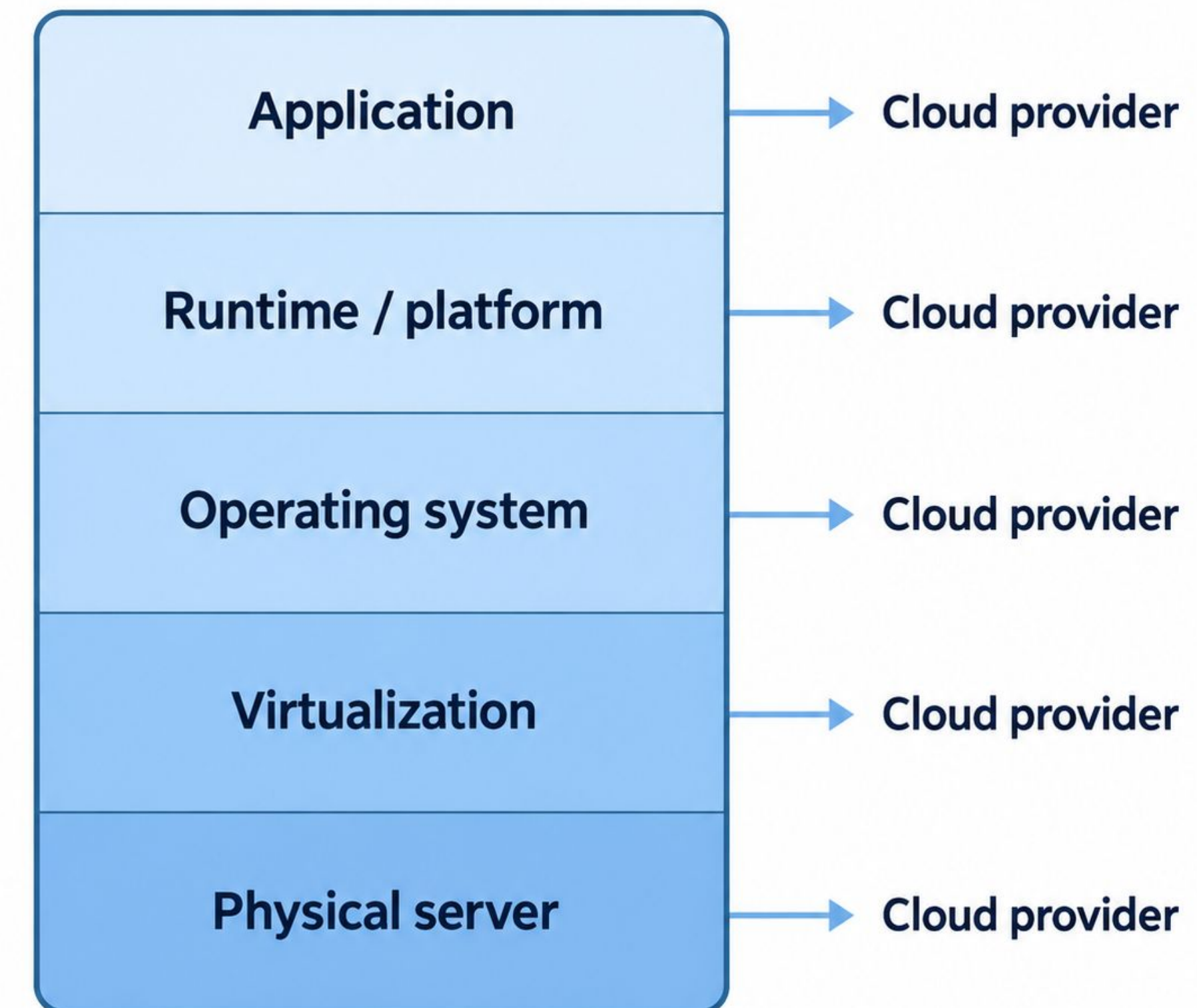
User mainly manages:

- account/settings
- application configuration
- their data/content

Usually accessed through a browser or client application

Example:
Google Workspace
Microsoft 365
Salesforce

SaaS



You mainly manage: account / settings / data

Compare IaaS / PaaS / SaaS

	IaaS	PaaS	SaaS
Main idea	Rent infrastructure	Deploy applications	Use finished software
You manage	OS, runtime, app, data	App, data/config	Account, settings, data
Provider manages	Hardware, virtualization	Infrastructure, OS, runtime	Almost entire stack
Control	High	Medium	Low
Management effort	High	Medium	Low
Example	Compute Engine	App Engine	Google Workspace

More control ← IaaS — PaaS — SaaS → **Less infrastructure to manage**

Practice: Choose the Right Cloud Service Model

Scenario: For each scenario, choose the most appropriate model: IaaS, PaaS, FaaS/Serverless, or SaaS. Briefly explain why.

- **A.** Your company needs to migrate a legacy application that requires root access, a custom OS configuration, and special system libraries.
- **B.** Whenever a user uploads an image to Cloud Storage, you want a small piece of code to automatically generate a thumbnail.
- **C.** Your department wants an online collaboration platform for email, documents, meetings, and file sharing, without developing the software itself.
- **D.** Your team has built a Python web application and wants automatic deployment and scaling without managing servers or the operating system.



Summary

- **IaaS** — manage more infrastructure; gain more control
- **PaaS** — focus on the application; provider manages the platform
- **FaaS / Serverless** — run small pieces of code when triggered
- **SaaS** — use a complete application managed by the provider
- Cloud services exist on a **spectrum of control vs. management responsibility**
- Choose the model based on:
 - required control
 - operational effort
 - application needs

There is no single “best” service model — **choose the one that fits the problem.**